

Implementação e Análise de um Pipeline de 5 Estágios com Resolução de Hazards na Arquitetura RISC-V

Felipe Barros Ratton de Almeida¹, Laura Menezes Heráclito Alves¹, Mateus Ribeiro Fernandes¹, Vitor de Meira Gomes¹

¹ICEI — Instituto de Ciências Exatas e Informática, PUC Minas, Brasil

Corresponding author: felipebarrosratton.almeida@gmail.com, laura.heraclito@gmail.com, mateusxirimba@gmail.com, vitormeiragomes@outlook.com

Abstract. Este trabalho apresenta a implementação de um pipeline de 5 estágios baseado na arquitetura RISC-V, com foco na resolução de hazards de dados e controle. Foram desenvolvidas soluções para desvios condicionais, incluindo flush do pipeline, além de técnicas como forwarding e inserção de stalls. Os resultados obtidos demonstram o impacto dessas técnicas no desempenho do processador, analisado por meio do CPI. A implementação foi validada por meio de simulações, evidenciando a correta execução das instruções e a melhoria no fluxo de dados.

Key words: RISC-V; Pipeline; Hazards; Forwarding; Computer Architecture

Received: **Revised:** **Accepted:**

1. Introdução

A técnica de pipeline é amplamente utilizada em arquiteturas modernas para aumentar o paralelismo e a vazão de execução de instruções. Ao dividir o processamento em múltiplos estágios — Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM) e Write Back (WB) — torna-se possível sobrepor a execução de diferentes instruções em um mesmo intervalo de tempo.

Entretanto, essa abordagem introduz desafios significativos relacionados à consistência dos dados e ao controle do fluxo de execução. Entre esses desafios, destacam-se os hazards de dados (RAW - Read After Write) e os hazards de controle, que podem comprometer tanto a corretude quanto o desempenho do sistema.

Este trabalho tem como objetivo implementar e analisar um pipeline de 5 estágios baseado na arquitetura RISC-V, abordando progressivamente a resolução de hazards por meio de técnicas como inserção de NOPs, forwarding (bypassing) e detecção automática de dependências. Além da implementação funcional, investiga-se o impacto dessas soluções no desempenho, utilizando métricas como ciclos totais e CPI (Cycles Per Instruction).

2. Desenvolvimento e Implementação

2.1. Parte 1: Branch e Controle de Fluxo

2.1.1 Análise do Problema

Inicialmente, o processador não tratava corretamente instruções de desvio condicional, especificamente a instrução *BEQ* (*Branch if Equal*). Esse problema ocorre porque, em uma arquitetura pipeline, múltiplas instruções são processadas simultaneamente em diferentes estágios. Assim, quando a decisão do branch é resolvida no estágio EX, as instruções subsequentes já se encontram parcialmente carregadas no pipeline.

No programa de teste utilizado, a instrução *beq x4, x4* é sempre verdadeira, ou seja, o desvio deve sempre ser tomado. Consequentemente, as instruções imediatamente seguintes deveriam ser descartadas. No entanto, sem um mecanismo adequado de controle, essas instruções continuavam sendo executadas, alterando indevidamente o estado dos registradores e da memória.

Esse comportamento caracteriza um *hazard de controle*, em que o fluxo correto do programa não é respeitado devido à latência na resolução do desvio.

2.1.2 Implementação

A solução foi implementada no módulo *BranchUnit*, onde a avaliação da condição de desvio ocorre no estágio EX do pipeline. Nesse estágio, os valores dos registradores fonte são comparados e, caso sejam iguais, o sinal *branch_taken* é ativado.

O endereço de destino do branch é calculado a partir da soma do valor do contador de programa no estágio de execução com o imediato da instrução:

$$branch_target = pc_ex + branch_imm$$

Quando o branch é tomado, o valor do PC é atualizado com o endereço de destino e as instruções que já se encontram nos estágios anteriores do pipeline (IF/ID e ID/EX) são invalidadas por meio de um mecanismo de *flush*. Esse processo consiste em substituir essas instruções por NOPs, impedindo sua execução.

2.1.3 Validação Experimental

A validação foi realizada por meio de simulação utilizando o ambiente fornecido, com análise dos sinais no GTKWave. Foi possível observar o instante em que o sinal *branch_taken* é ativado, bem como a atualização do PC para o endereço correto de destino.

As métricas obtidas durante a execução foram:

- Ciclos totais: 14
- Instruções executadas: 7
- Branches: 1
- Flushes: 1
- Stalls: 0
- Bypasses: 0
- CPI: 2.00

A presença de um único *flush* confirma que o mecanismo de controle de fluxo foi acionado corretamente quando o branch foi tomado.

2.1.4 Evidência por Forma de Onda

2.1.5 Limitações Observadas

Apesar da implementação correta do controle de fluxo, os resultados finais da execução ainda apresentaram inconsistências, como valores incorretos em registradores e memória. Exemplos observados incluem:

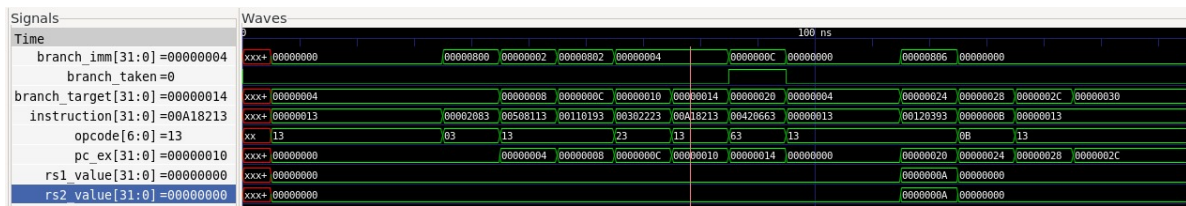


Figure 1: Sinais do BranchUnit no GTKWave mostrando *branch_taken* e o update do PC.

- x2 esperado 15, obtido 5
- x3 esperado 16, obtido 1
- x4 esperado 26, obtido 10

Esses erros decorrem de hazards RAW não tratados (ausência de forwarding e detecção de dependências), e serão resolvidos nas Partes 2 e 3.

2.1.6 Impacto no Pipeline

A introdução de instruções de desvio condicional afeta diretamente o desempenho do pipeline. No caso analisado, a necessidade de descartar instruções já carregadas resultou na inserção de um *flush*, aumentando o número total de ciclos.

Esse comportamento explica o valor de CPI igual a 2.00, superior ao ideal ($CPI = 1$). A penalidade observada evidencia o custo associado ao controle de fluxo em arquiteturas pipeline, justificando a necessidade de técnicas mais avançadas, como predição de desvios, para otimização do desempenho.

2.1.7 Relação com a Análise Esperada

Nesta etapa, ainda não há forwarding nem detecção automática de hazards. Por isso, *stalls* não são inseridos pelo hardware e as dependências de dados permanecem sem tratamento. O impacto do branch já aparece pelo *flush* e pelo aumento do CPI, evidenciando a penalidade de controle no pipeline.

2.2. Parte 2: Execução sem Tratamento de Dependências

2.2.1 Objetivo

Nesta etapa, o pipeline ainda não possui mecanismos de detecção ou resolução de hazards de dados. Dessa forma, instruções com dependências do tipo RAW (Read After Write) resultam em execuções incorretas.

O objetivo foi adaptar o código assembly manualmente, inserindo instruções NOP para eliminar essas dependências e garantir a correteza da execução.

2.2.2 Implementação

A solução adotada consistiu na modificação da rotina *load_program_full_dependencies*, inserindo instruções NOP entre pares de instruções dependentes.

Essa abordagem garante que o valor produzido por uma instrução esteja disponível no banco de registradores antes de ser consumido por outra.

Sem forwarding ou hazard detection, o dado só se torna válido após o estágio de Write Back (WB). Portanto, a instrução consumidora só pode ler o valor corretamente no estágio ID após esse momento.

Como consequência, foi necessário inserir três NOPs entre o produtor e o consumidor.

2.2.3 Mapeamento de Dependências

A Tabela 1 apresenta o mapeamento das dependências identificadas e a quantidade de bolhas inseridas.

Table 1: Mapeamento de bolhas manuais (Etapa 2)

Produtor	Consumidor	Dependência	NOPs
lw x1	addi x2	RAW em x1	3
addi x2	addi x3	RAW em x2	3
addi x3	sw x3	RAW em x3	3
addi x3	addi x4	RAW em x3	0
addi x4	beq x4	RAW em x4	3

Observa-se que nem todas as dependências exigem inserção adicional de NOPs, pois algumas já são naturalmente resolvidas devido ao espaçamento introduzido por instruções intermediárias.

2.2.4 Validação Experimental

A execução do programa modificado apresentou os seguintes resultados:

- Ciclos totais: 26
- Instruções executadas: 7
- Stalls: 0
- Bypasses: 0
- Branches: 1
- Flushes: 1
- CPI: 3.71

Diferentemente da etapa anterior, o programa agora executa corretamente, conforme evidenciado pelo resultado:

PASS: all expected results match

Os valores finais dos registradores e da memória confirmam a corretude da execução.

2.2.5 Evidência por Forma de Onda

A forma de onda confirma o comportamento esperado: o opcode do BEQ (0x63) aparece no estágio EX, o sinal *branch_taken* é ativado no mesmo ciclo e o *flush* ocorre em seguida. Ao longo do trecho, os registradores de pipeline exibem vários 0x00000013, evidenciando as bolhas inseridas manualmente para evitar dependências RAW.

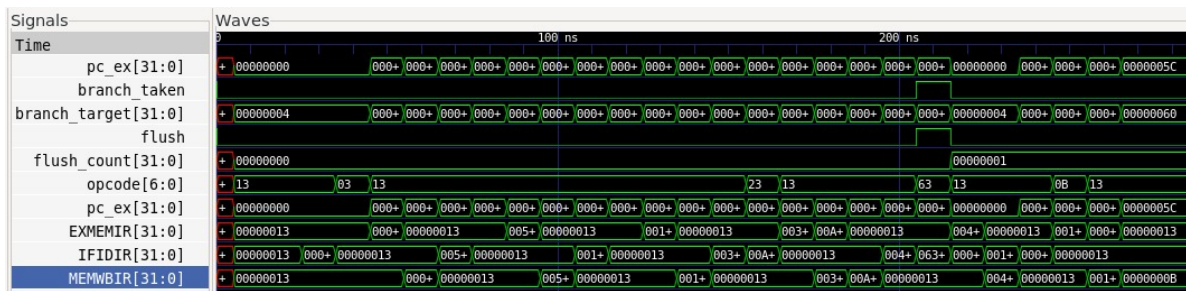


Figure 2: Waveform da Parte 2 mostrando NOPs (0x00000013) nos registradores de pipeline e o branch sendo tomado.

2.2.6 Análise do Pipeline

A inserção manual de NOPs introduz bolhas no pipeline, que podem ser observadas diretamente no GTKWave através de instruções com valor 0x00000013, correspondente ao NOP.

Além disso, foi possível verificar:

- O sinal *branch_taken* sendo ativado corretamente
- Atualização do *branch_target*
- Ocorrência de *flush* após o branch

Esses sinais confirmam que o controle de fluxo permanece funcional mesmo após a inserção das bolhas.

2.2.7 Impacto no Desempenho

Apesar da correteza da execução, o desempenho do pipeline foi significativamente degradado.

O CPI elevado (3.71) é consequência direta do aumento no número de ciclos sem aumento proporcional no número de instruções úteis. Isso ocorre porque os NOPs consomem ciclos de clock, mas não realizam nenhuma operação útil.

Esse comportamento evidencia a ineficiência da abordagem baseada em software.

2.2.8 Relação com a Análise Esperada

Nesta etapa não há forwarding nem detecção automática de hazards, portanto *stalls* não são inseridos pelo hardware. As bolhas são inseridas manualmente via NOPs para evitar RAW. O forwarding resolveria dependências quando o valor produzido está em MEM ou WB e pode ser encaminhado para a ALU no EX, evitando essas bolhas (o que será implementado na Parte 3). O impacto do branch continua visível pelo *flush* e pelo aumento do CPI.

2.2.9 Conclusão da Etapa

A inserção manual de NOPs resolve completamente os problemas de dependência de dados, garantindo a execução correta do programa.

Entretanto, essa solução apresenta um alto custo em termos de desempenho, tornando evidente a necessidade de mecanismos mais sofisticados, como forwarding e detecção automática de hazards, que serão implementados na próxima etapa.

2.3. Parte 3: Hazard Detection e Forwarding

2.3.1 Objetivo

Nesta etapa, o objetivo foi implementar no hardware a resolução de dependências de dados, eliminando a necessidade de NOPs manuais. Para isso, foram implementadas: (i) uma Forwarding Unit para encaminhar resultados diretamente para a ALU e (ii) uma Hazard Detection Unit para detectar o caso de load-use e inserir um stall apenas quando necessário.

2.3.2 Implementação

Forwarding Unit: a seleção de operandos foi feita separadamente para *rs1* e *rs2*, com prioridade para o resultado mais recente. A ordem adotada foi: (1) EX/MEM para resultado de ALU, (2) MEM/WB para resultado de ALU e (3) MEM/WB para resultado de load. O registrador x0 foi ignorado para evitar dependências falsas.

Hazard Detection Unit: foi implementada a detecção de load-use. A unidade identifica se a instrução em EX/MEM é um `lw` e se a instrução em ID/EX utiliza o mesmo registrador de destino. Para isso, o uso de *rs1* e *rs2* é definido por opcode: `lw` e `addi` usam apenas *rs1*, enquanto `sw` e `beq` usam *rs1* e *rs2*. Quando o caso ocorre, um stall é inserido por um ciclo.

2.3.3 Validação Experimental

Dois programas foram executados:

1) `full_dependencies` (com NOPs):

- Ciclos totais: 26
- Instruções executadas: 7
- Stalls: 0
- Bypasses: 0
- Branches: 1
- Flushes: 1
- CPI: 3.71

PASS: all expected results match.

2) `forwarding_and_stalls` (sem NOPs):

- Ciclos totais: 18
- Instruções executadas: 11
- Stalls: 2
- Bypasses: 6
- Branches: 0
- Flushes: 0
- CPI: 1.64

PASS: forwarding and stalls behaved as expected.

Os valores finais de registradores e memória confirmam a execução correta, com dependências resolvidas automaticamente.

2.3.4 Evidência por Forma de Onda

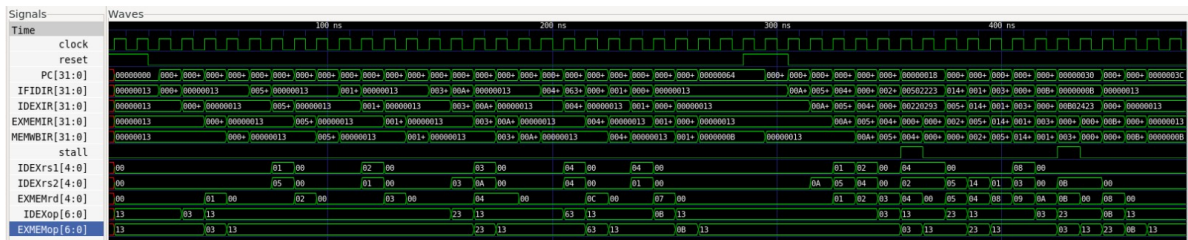


Figure 3: Waveform da Parte 3 mostrando o avanço das instruções no pipeline e a ocorrência de stall em casos de load-use.

A primeira forma de onda evidencia o sinal *stall* sendo ativado exatamente quando uma instrução tenta usar o resultado de um *lw* ainda não disponível. Nesse ciclo, o pipeline é segurado e uma bolha é inserida, confirmando a atuação da Hazard Detection Unit.

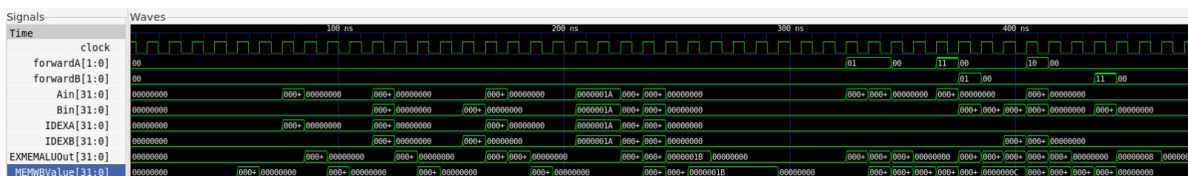


Figure 4: Waveform da Parte 3 destacando os sinais *forwardA/forwardB* e a seleção de operandos encaminhados.

A segunda forma de onda destaca *forwardA* e *forwardB* assumindo valores diferentes de 00, indicando encaminhamento de EX/MEM e MEM/WB. Os sinais *Ain* e *Bin* mostram que a ALU recebe os valores encaminhados, confirmando o funcionamento da Forwarding Unit.

2.3.5 Análise do Pipeline

Por que stalls ocorrem: o stall é necessário apenas no caso de load-use, pois o dado carregado por *lw* só fica disponível após MEM. Assim, a instrução consumidora precisa esperar um ciclo para usar o valor.

Quando forwarding resolve dependências: sempre que o resultado já foi calculado (EX/MEM) ou está em MEM/WB, a Forwarding Unit encaminha diretamente para a ALU, evitando bolhas.

Impacto do branch no desempenho: o controle de fluxo continua exigindo *flush* quando o branch é tomado, mantendo a penalidade de controle observada na Parte 1.

2.3.6 Impacto no Desempenho

Com o forwarding, o CPI foi reduzido de 3.71 (etapa com NOPs manuais) para 1.64 no teste sem NOPs, mostrando melhoria significativa. As bolhas ficaram restritas aos casos inevitáveis de load-use (2 stalls), enquanto a maioria das dependências foi resolvida sem perda de desempenho (6 bypasses).

2.3.7 Conclusão da Etapa

A etapa mostrou que a resolução automática de hazards reduz drasticamente o número de bolhas e melhora o desempenho, mantendo a execução correta. O forwarding eliminou a maior parte das dependências, e o stall foi inserido apenas quando necessário, atendendo plenamente aos requisitos da Parte 3.

3. Análise Crítica de Desempenho

A avaliação do desempenho do processador antes e após a implementação da resolução de hazards em hardware demonstra um ganho significativo de eficiência operacional. Para garantir consistência experimental, compararam-se dois cenários distintos: (i) execução com dependências resolvidas via software (inserção de NOPs) e (ii) execução com resolução automática em hardware (forwarding e stalls).

Table 2: Comparativo Geral de Métricas do Pipeline

Métrica	Solução Software (NOPs)	Solução Hardware (Final)
Ciclos Totais	26	18
Instruções Executadas	7	11
Bypasses Utilizados	0	6
Stalls Automáticos	0	2
CPI Final	3.71	1.64

3.1. Variável 1: Cálculo de Speedup (Ganho de Velocidade)

A substituição das bolhas de software (*NOPs*) pelo roteamento dinâmico de dados (*Forwarding*) reduziu a contagem total de ciclos de execução de 26 para 18. O *Speedup* (S) é calculado pela relação entre o tempo de execução original e o novo:

$$S = \frac{Tempo_{antigo}}{Tempo_{novo}} = \frac{26}{18} \approx 1,44 \quad (1)$$

Este resultado indica que o processador otimizado em hardware é aproximadamente **44% mais rápido** em termos de ciclos totais quando comparado à abordagem baseada em NOPs.

3.2. Variável 2: Otimização do CPI (Ciclos por Instrução)

O limite teórico de desempenho em um pipeline ideal é um CPI de 1,0.

- Na **abordagem via software**, o CPI atingiu 3,71, evidenciando o elevado custo das bolhas inseridas manualmente.
- Na **abordagem em hardware**, o CPI foi reduzido para **1,64**, aproximando-se significativamente do ideal.

Essa redução demonstra que a maior parte das dependências de dados foi resolvida sem a necessidade de interrupções no fluxo do pipeline. O valor final não atinge 1,0 devido a dois fatores principais: o preenchimento inicial do pipeline (*pipeline fill*) e a ocorrência inevitável de stalls no caso específico de dependências do tipo *load-use*.

3.3. Variável 3: Eficiência dos Mecanismos de Hardware

A eficácia da solução implementada pode ser analisada a partir dos eventos registrados durante a execução:

- **ForwardingUnit:** O registro de 6 *bypasses* indica que a unidade foi amplamente utilizada para encaminhar dados diretamente aos operandos da ALU, eliminando a necessidade de ciclos de espera.
- **Hazard Detection Unit:** A ocorrência de apenas 2 *stalls* confirma que o hardware interveio exclusivamente nos casos em que o forwarding não era suficiente, especificamente em dependências do tipo *load-use*.

Esses resultados evidenciam que a arquitetura foi capaz de maximizar o paralelismo do pipeline, limitando as interrupções apenas às situações estritamente necessárias.

Conclui-se que o *trade-off* entre o aumento da complexidade do hardware e o ganho de desempenho obtido é altamente favorável, uma vez que a solução elimina a necessidade de NOPs manuais e melhora significativamente a eficiência da execução.

4. Conclusão

Este trabalho permitiu analisar, de forma prática, os impactos dos hazards em uma arquitetura pipeline de 5 estágios baseada em RISC-V. A implementação progressiva das soluções evidenciou claramente a diferença entre um pipeline funcional sem otimização e um pipeline eficiente com suporte a mecanismos de hardware.

A abordagem inicial, baseada na inserção manual de instruções *NOP*, garantiu a correteza da execução ao eliminar dependências do tipo RAW, porém introduziu um alto custo de desempenho, refletido em um CPI de 3.71 e em um grande número de ciclos ociosos.

Por outro lado, a implementação das unidades de *Forwarding* e *Hazard Detection* possibilitou a resolução automática dessas dependências diretamente no hardware. O encaminhamento de dados eliminou a maior parte das bolhas, enquanto a inserção de *stalls* ocorreu apenas nos casos inevitáveis de conflito do tipo *load-use*.

Como resultado, observou-se uma redução significativa no número de ciclos totais (de 26 para 18) e no CPI (de 3.71 para 1.64), além de um ganho de desempenho de aproximadamente 1,44x. Esses dados demonstram que a arquitetura proposta foi capaz de manter a correteza da execução ao mesmo tempo em que aumentou significativamente a eficiência do pipeline.

Conclui-se que a utilização de mecanismos de resolução de hazards em hardware é fundamental para explorar o paralelismo do pipeline de forma eficiente. O equilíbrio entre complexidade adicional do hardware e ganho de desempenho mostrou-se favorável, consolidando essas técnicas como essenciais no projeto de processadores modernos.

References

- [1] Donald E. Knuth. *The T_EXbook*. Addison-Wesley, 1984.